University of Southern Denmark

Department of Mathematics and Computer Science

SPD801: Master Thesis in Computer Science

Formalising πLL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

13th May 2026

1 Introduction

The Curry-Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π-calculus.

This line of work has been further developed by [Montesi and Peressotti 2021], who provide a unified account reconciling linear logic, the π-calculus, and their metatheory. Their framework serves as the formal basis for the system πLL considered in this thesis.

Motivation and correctness concerns. Despite the elegance of these theoretical correspondences, their informal and paper-based presentations are prone to subtle errors. In particular, the application of inference rules in linear logic and session-typed systems often relies on implicit ordering conventions and intuitive interpretations of syntax. This can lead to mistakes that are not immediately apparent in written derivations.

As an illustrative example, small inconsistencies can arise in the presentation of typing rules within such systems, where the intuitive alignment between variable ordering and type assignment does not precisely match the formal rule structure. These discrepancies are not flaws of the underlying theory, but rather reflect the difficulty of maintaining precision in informal presentations, including in the framework of Montesi and Peressotti.

As observed in discussions with the authors of the framework, such issues are easy to overlook because rule application is often guided by intuition rather than strict syntactic structure. This highlights that even formally correct rules may be unintuitive to apply due to their representation.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanization. Mechanization provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanization in this work is to validate and internalize existing results within a proof assistant. This allows us to eliminate ambiguity arising from informal presentation and to ensure that all derivations are checked with respect to a precise formal semantics.

Moreover, mechanized developments have repeatedly demonstrated their ability to uncover small but meaningful inconsistencies in paper proofs, supporting the view that proof assistants serve as a valuable tool for verifying, rather than merely implementing, theoretical frameworks.

Contributions. In this thesis, we present a mechanized formalization of the π-calculus-based linear logic system πLL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalization of the syntax and typing system of πLL in Lean, including processes, session types, and substitution operations.
- A formalization of the operational semantics for the multiplicative fragment of πLL.
- Formal proofs of session fidelity for the multiplicative fragment.
- A locally nameless representation of binding, eliminating the need for reasoning up to α-equivalence and simplifying substitution reasoning.

Scope of the formalization. This thesis focuses on a mechanized reconstruction of the πLL system of [Montesi and Peressotti 2021], with particular emphasis on its multiplicative fragment. The

syntactic and typing components of πLL have been fully mechanized, including processes, session types, typing judgments, and substitution operations. The operational semantics are defined for the multiplicative fragment, with the exception of the res-rule.

Consequently, metatheoretic results concerning typing apply to the full system, while results involving operational semantics are restricted to the multiplicative fragment. The aim of this work is not to extend the theory, but to provide a mechanically verified account of an existing theoretical framework.

2 Background

This section begins by introducing the theoretical foundations underlying the formalisation following the presentation of [Montesi and Peressotti 2021], We cover the full πLL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment πMLL when discussing operational semantics.

Following [Montesi and Peressotti 2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

In particular, conjunction and disjunction split into multiplicative and additive variants. The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \parr B$, together with the corresponding units 1 and $\bot$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the joint availability of two independent resources, while $A \parr B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B, whereas $A \parr B$ corresponds to receiving a value of type A and proceeding as B. The units 1 and $\bot$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1}\,1 \qquad \frac{\vdash \Gamma}{\vdash \Gamma, \bot}\,\bot \qquad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B}\,\otimes \qquad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \parr B}\,\parr$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B, each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B.

The unit rule rule 1 introduces the unit 1, which represents the absence of further obligations. It can be understood as an "empty output", corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, rules $\bot$ and $\parr$ describe the behavior of the connective $A \parr B$ and its unit $\bot$. The $\parr$ rule combines resources within a single context, reflecting a form of interaction where both components

are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B. The unit $\bot$ represents the absence of further input, acting as an "empty input" and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the rule 1 and combined via rule $\otimes$ to derive $1 \otimes 1$:

$$\dfrac{\dfrac{}{\vdash 1}\,1 \quad \dfrac{}{\vdash 1}\,1}{\vdash 1 \otimes 1}\otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-oriented view of logic provides the foundation for its applications in computer science, particularly in the study of concurrency and session-typed communication. In the following sections, this perspective is used to motivate the process interpretation underlying πLL.

2.2 The π-Calculus: Processes and Communication

The π-calculus [Milner et al. 1992] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of Montesi and Peressotti [2021], which adopts Wadler's convention of using square brackets '[-]' for output and round parentheses '(-)' for input.

Programs in πMLL are processes (P, Q, R, ...), which communicate over names (x, y, z, ...) representing session endpoints. Sessions are binary (they involve exactly two endpoints), a discipline enforced by the typing system introduced in 2.3. We assume an infinite set of names with decidable equality. The process terms of πMLL are defined by the following grammar:

$P, Q ::=$	$x[y].P$	*output y on x and continue as P*
$\mid$	$x(y).P$	*input y on x and continue as P*
$\mid$	$x[].P$	*output (empty message) on x and continue as P*
$\mid$	$x().P$	*input (empty message) on x and continue as P*
$\mid$	$\nu xy\,P$	*name restriction (cut)*
$\mid$	$P \mid Q$	*parallel composition*
$\mid$	$x[\mathrm{L}].P$	*select left on x and continue as P*
$\mid$	$x[\mathrm{R}].P$	*select right on x and continue as P*
$\mid$	$x.\mathrm{case}\{\mathrm{L}:P, \mathrm{R}:Q\}$	*offer a binary choice between P or Q on x*
$\mid$	$x[A].P$	*output type A on x and continue as P*
$\mid$	$x(X).P$	*input a type as X on x and continue as P*
$\mid$	$!x\langle z_1, \ldots, z_n\rangle.\{P\}$	*server (replicable process) on x depending on servers on $z_1 \ldots z_n$*
$\mid$	$x[\mathrm{USE}].P$	*consume a server on x and continue as P*
$\mid$	$x[\mathrm{DUP}](x').P$	*duplicate server x as x' in P*
$\mid$	$x[\mathrm{DISP}].P$	*dispose of a server on x*
$\mid$	$x \leftrightarrow y$	*link x and y*
$\mid$	$\mathbf{0}$	*terminated process*

Term $x[y].P$ allocates a fresh name y, outputs it over x, and proceeds as P. Dually, $x(y).P$ inputs a name over x, binding it to y in P. Terms $x[].P$ and $x().P$ model empty output and input.

Restriction $\nu xy\,P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\nu xy\,P = \nu yx\,P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and $\mathbf{0}$ is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0})$. This process creates a private session between the restricted endpoints x and z. The left process, $x[].\mathbf{0}$, signals termination on x before terminating, while the right process, $z().y[].\mathbf{0}$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for πLL. In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and Propositions-as-Sessions Correspondence

In πLL, session types are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality, the key structural feature of classical linear logic, ensures that the two endpoints of every session implement complementary protocols.

The correspondence between linear logic and session types as presented in [Montesi and Peressotti 2021] following [Carbone et al. 2016; Wadler 2014] is captured directly by the type grammar. Each connective denotes a communication action, where $A \otimes B$ describes sending a value of type A and continuing as B, while its dual $A \parr B$ describes receiving. The units $\mathbf{1}$ and $\bot$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \,\&\, B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of πLL is defined as follows:

$A, B ::=$	$A \otimes B$	*send A, proceed as B*	$\mid$	$A \parr B$	*receive A, proceed as B*
$\mid$	$\mathbf{1}$	*empty output, unit for* $\otimes$	$\mid$	$\bot$	*empty input, unit for* $\parr$
$\mid$	$A \,\&\, B$	*offer A or B*	$\mid$	$A \oplus B$	*select A or B*
$\mid$	$\exists X.A$	*existential, type output*	$\mid$	$\forall X.A$	*universal, type input*
$\mid$	$?A$	*client request*	$\mid$	$!A$	*server accept*
$\mid$	X	*type variable*	$\mid$	$X^{\bot}$	*dual of type variable*

Duality is defined structurally on session types by exchanging each connective with its dual and forms an involution, i.e. $(A^{\bot})^{\bot} = A$. Consequently, the endpoints of a well-typed session are always assigned dual types, ensuring they are compatibility:

$$(A \otimes B)^{\bot} = A^{\bot} \parr B^{\bot} \qquad \mathbf{1}^{\bot} = \bot \qquad (A \oplus B)^{\bot} = A^{\bot} \,\&\, B^{\bot}$$
$$(!A)^{\bot} = ?A^{\bot} \qquad (\exists X.A)^{\bot} = \forall X.A^{\bot} \qquad (X)^{\bot} = X^{\bot}$$

2.4 Environments, Hyperenvironments and Judgements

Typing in πLL is organised into two layers: environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in [Montesi and Peressotti 2021], *environments* ($\Gamma, \Delta, \Xi, \ldots$) are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A, \ldots, x_n : A_n$, where each x_i is associated to its respective A_i, for $1 \leq i \leq n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then

Γ, Δ is the environment containing exactly the associations in Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using ',' as the sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \qquad \Gamma, \Delta = \Delta, \Gamma \qquad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by handled by *hyperenvironments* ($\mathcal{G}, \mathcal{H}, \mathcal{I}, \ldots$), which are collections of environments joined using the parallel operator '$\mid$'. They can be written as follows $\mathcal{G} = \Gamma_1, \ldots, \Gamma_n$, where $\Gamma_1, \ldots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} \mid \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using '$\mid$' the sum and $\varnothing$ as unit, where $\varnothing$ is the hyperenvironment containing no environments:

$$\mathcal{G} \mid \varnothing = \mathcal{G} \qquad \mathcal{G} \mid \mathcal{H} = \mathcal{H} \mid \mathcal{G} \qquad (\mathcal{G} \mid \mathcal{H}) \mid \mathcal{I} = \mathcal{G} \mid (\mathcal{H} \mid \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations ($\mathcal{D}, \mathcal{E}, \mathcal{F}, \ldots$) are written as $\vdash P :: \mathcal{G}$ $^{\mathcal{D}}$ and are read as the derivation $\mathcal{D}$ having the judgement $\vdash P :: \mathcal{G}$ as its conclusion. The meaning of this statement is that the process, P, appearing in the judgement concluding a derivations is well-typed.

Multiplicative Fragment

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\ \text{CUT} \qquad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \qquad \frac{}{\vdash \mathbf{0} :: \varnothing}\ \text{MIX}_0$$

$$\frac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B}\ \otimes \qquad \frac{\vdash P :: \varnothing}{\vdash x[].P :: x : 1}\ 1 \qquad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \parr B}\ \parr \qquad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \bot}\ \bot$$

Additional rules for Additives, Quantifiers and Exponentials

$$\frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{L}].P :: \Gamma, x : A \oplus B}\ \oplus_1 \qquad \frac{\vdash P :: \Gamma, x : B}{\vdash x[\text{R}].P :: \Gamma, x : A \oplus B}\ \oplus_2 \qquad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\text{case}\{\text{L} : P, \text{R} : Q\} :: \Gamma, x : A \,\&\, B}\ \&$$

$$\frac{}{\vdash x \leftrightarrow y :: x : A^{\perp}, y : A}\ \text{AX} \qquad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A}\ ? \qquad \frac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A}\ \text{W} \qquad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A}\ \text{C}$$

$$\frac{\vdash P :: z_1 : ?B_1, \ldots, z_n : ?B_n, x : A}{\vdash\, !x\langle z_1, \ldots, z_n\rangle.\{P\} :: z_1 : ?B_1, \ldots, z_n : ?B_n, x : !A}\ ! \qquad \frac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B}\ \exists \qquad \frac{\vdash P :: \Gamma, x : B \quad X \notin \text{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B}\ \forall$$

Fig. 1. πLL, typing rules.

The multiplicative rules form the core of the system. Rule CUT composes two processes in P sharing dual endpoints $x : A$ and $y : A^{\perp}$, hiding them via restriction. This is the logical equivalent of communication. Rule MIX and rule MIX_0 compose independent processes in parallel and introduce the terminated process, respectively.

Rule $\otimes$ and rule $\parr$ type output and input. Note that rule $\otimes$ types the sent name y in a *separate* environment from the continuation, reflecting that the two are independent resources. By contrast, rule $\parr$ keeps y and the continuation x in the *same* environment, reflecting their sequential dependency. The units rule 1 and rule $\bot$ type empty output and empty input, where rule 1 requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to $\mathbf{0}$.

The additive rules (rule $\oplus_1$, rule $\oplus_2$, and rule $\&$) type selection and offering of alternative processing paths. Note that rule $\&$ requires both branches to be typed in the *same* context Γ, reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The rule ! requires all free names except x to be typed using ?, enforcing that a server only depends on other servers. The rule ?, rule W, and rule C allow a client to use, discard, or duplicate a server respectively.

The rule AX types a link between two dual endpoints, $x \leftrightarrow y$, forwarding all messages sent on x safely to y and vice versa. Rule $\exists$ and rule $\forall$ type polymorphic communication, where the side condition $X \notin \mathrm{ft}(\Gamma)$ in rule $\forall$, ensures the introduced type variable, X, does not accendentally shadow one already existing in Γ.

Example 2.2. The process from Example 2.1 has the typing $\vdash \boldsymbol{\nu} zx\,(z().y[].\mathbf{0} \mid x[].\mathbf{0}) :: y : 1$, as shown by the derivation below:

$$\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\dfrac{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}{\vdash \boldsymbol{\nu} xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\,\text{CUT}}\,\text{MIX}$$

Note that, for the typing context $x : 1 \mid y : 1, z : \bot$ to align with the premise of rule CUT, we utilize the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \varnothing$ and $\Gamma = \bullet$. Under these instantiations, the premise of rule CUT: $\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}$ can be viewed as $x : A \mid \Delta, y : A^{\perp}$. This effectively instantiates the CUT over the dual types $A = 1$ and $A^{\perp} = \bot$ on channels x and y, with $\Gamma = y : 1$, while ensuring the restricted endpoints implement dual session behaviours, fulfilling the requirements for a creating a valid CUT (session).

2.5 Operational Semantics

The operational semantics of πLL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in Figure 2. The full LTS is given in Montesi and Peressotti [2021].

Remark 2.3. The operational semantics rely explicitly on the structural equivalence rule (rule $=_\alpha$) to rename restricted variables (via $=_\alpha$) prior to a transition. In paper-based proofs, α-equivalence is often treated implicitly or swept under the rug via Barendregt's Variable Convention. However, in a mechanized setting, explicitly reasoning about α-equivalence over named variables is notoriously difficult and introduces overwhelming boilerplate, as variable capture must be manually avoided during substitution [**??**]. As we will see in **??**, this challenge motivates the use of a Locally Nameless

$$\frac{}{\vdash P :: \varnothing,\ x:1}\ 1 \quad \xrightarrow{x[]} \quad \vdash P :: \varnothing$$

$$\frac{\vdash P :: \Gamma, x':A \mid \Delta, x:B}{\vdash x[x'].P :: \Gamma, \Delta, x:A \otimes B}\ \otimes \quad \xrightarrow{x[x']} \quad \vdash P :: \Gamma, x':A \mid \Delta, x:B$$

$$\frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x:\bot}\ \bot \quad \xrightarrow{x()} \quad \vdash P :: \Gamma$$

$$\frac{\vdash P :: \Gamma, x':A, x:B}{\vdash x(x').P :: \Gamma, x:A \parr B}\ \parr \quad \xrightarrow{x(x')} \quad \vdash P :: \Gamma, x':A, x:B$$

$$\frac{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash P' :: \mathcal{G}' \qquad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l} \dfrac{\vdash P' :: \mathcal{G}' \quad \vdash Q :: \mathcal{H}}{\vdash P' \mid Q :: \mathcal{G}' \mid \mathcal{H}}\ \text{MIX}}\ \text{PAR}_1$$

$$\frac{\vdash Q :: \mathcal{H} \xrightarrow{l} \vdash Q' :: \mathcal{H}' \qquad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l} \dfrac{\vdash P :: \mathcal{G} \quad \vdash Q' :: \mathcal{H}'}{\vdash P \mid Q' :: \mathcal{G} \mid \mathcal{H}'}\ \text{MIX}}\ \text{PAR}_2$$

$$\frac{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash P' :: \mathcal{G}' \qquad \vdash Q :: \mathcal{H} \xrightarrow{l'} \vdash Q' :: \mathcal{H}' \qquad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l|l'} \dfrac{\vdash P' :: \mathcal{G}' \quad \vdash Q' :: \mathcal{H}'}{\vdash P' \mid Q' :: \mathcal{G}' \mid \mathcal{H}'}\ \text{MIX}}\ \text{SYN}$$

$$\frac{P =_\alpha Q \qquad \vdash Q :: \mathcal{G} \xrightarrow{l} \vdash Q' :: \mathcal{G}'}{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash Q' :: \mathcal{G}'}\ =_\alpha \qquad \frac{\dfrac{\vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot \quad \xrightarrow{x[]\mid y()} \quad \vdash P' :: \mathcal{G} \mid \Gamma}{\vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot}\ 1\bot \quad \xrightarrow{\tau} \quad \vdash P' :: \mathcal{G} \mid \Gamma}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma}\ \text{CUT}$$

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp \quad \xrightarrow{x[x']\mid y(y')} \quad \vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\dfrac{\vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \text{CUT} \quad \xrightarrow{\tau} \quad \dfrac{\dfrac{\vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\vdash \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, \Xi, y:B^\perp}\ \text{CUT}}{\vdash \nu xy\, \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \text{CUT}}\ \otimes\parr$$

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp \xrightarrow{l} \vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp \qquad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\dfrac{\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\ \text{CUT} \xrightarrow{l} \dfrac{\vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp}{\vdash \nu xy\, P' :: \mathcal{G}' \mid \Gamma', \Delta'}\ \text{CUT}}\ \text{RES}$$

Fig. 2. πMLL, transition rules for derivations.

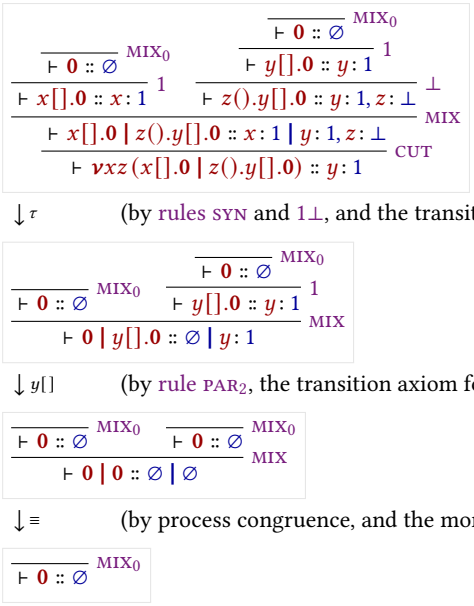

Fig. 3. An execution for the derivation in Example 2.2.

representation for the syntax of πLL, which structurally enforces α-equivalence and natively resolves the complexities of bound variable renaming.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l|l') \mid l, l' \in Act, \mathrm{i}(l) \cap \mathrm{i}(l') = \emptyset\}$. The restriction imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

Example 2.4. Section 2.5 shows a possible execution for the derivation of the process from Example 2.1.

Each logical rule (1, $\bot$, $\otimes$, $\parr$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule SYN allows for the pairing of concurrent actions via $l|l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules 1$\bot$ and $\otimes\parr$ simplify the term into an internal τ-transition, modeling the synchronization of the session.

2.6 Process Semantics and Erasure

While the typing derivations of πLL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π-calculus. To formally relate the typed derivations to their untyped execution, we follow [Montesi and Peressotti 2021] and define an erasure function, $proc : Der \to Proc$, which strips away the typing information from a derivation. Recursively, $proc$ maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any

well-typed πLL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash v[].\mathbf{0} :: v : 1}\,1}{\vdash w().v[].\mathbf{0} :: v : 1, w : \bot}\,\bot \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\text{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}\,\text{MIX}}{\vdash (w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v : 1, w : \bot \mid x : 1 \mid y : 1, z : \bot}\,\text{MIX}}{\vdash \nu xz\,(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v : 1, w : \bot \mid y : 1}\,\text{CUT}$$

Fig. 4. A derivation for the process $\nu xz\, w().y[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}$

Example 2.5. Consider the well-typed process P formed by adding a third parallel component to the derivation of Example 2.2 before the cut (A derivation of which can be seen on Figure 4):

$$P = \nu xz\,(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0})$$

The component containing w and v is independent from the component containing x, z, and y. This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$P \xrightarrow{\tau}\xrightarrow{w()}\xrightarrow{v[]}\xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{\tau}\xrightarrow{w()}\xrightarrow{y[]}\xrightarrow{v[]} \mathbf{0} \qquad P \xrightarrow{\tau}\xrightarrow{y[]}\xrightarrow{w()}\xrightarrow{v[]} \mathbf{0}$$

$$P \xrightarrow{w()}\xrightarrow{v[]}\xrightarrow{\tau}\xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{w()}\xrightarrow{\tau}\xrightarrow{v[]}\xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{w()}\xrightarrow{\tau}\xrightarrow{y[]}\xrightarrow{v[]} \mathbf{0}$$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated in [Montesi and Peressotti 2021], this relationship is established functorially. Let $\mathcal{P}(X) = \{X' \mid X' \subseteq X\}$ and $\mathcal{L}(X) = X \times Lbl$ be endofunctors representing the states and labeled transitions, respectively. The erasure function $proc$ acts as a homomorphism from LTS of derivations (lts_d) to the LTS of processes (lts_p), ensuring that the square of these LTS mappings commutes.

In the context of operational semantics, this functorial homomorphism yields an Erasure theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

Theorem 2.6 (Erasure). *The process LTS (lts_p) enjoys erasure with respect to the derivation LTS (lts_d). Concretely, for any valid typing derivation $\mathcal{D}$ and label $l \in Lbl$:*

(1) ***Soundness of Erasure:*** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$.*

(2) ***Completeness of Erasure:*** *If $proc(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $proc(\mathcal{D}') = P'$.*

Corollary 2.7 (Typability Preservation). *If P is well-typed and $P \xrightarrow{l} P'$, then P' is well-typed.*

This theorem allows us to define the operational target for πLL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

$$x[x'].P \xrightarrow{x[x']} P \qquad x(x').P \xrightarrow{x(x')} P \qquad x[].P \xrightarrow{x[]} P \qquad x().P \xrightarrow{x()} P$$

$$\frac{P \xrightarrow{l} P' \quad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q}\ \mathrm{PAR}_1 \qquad \frac{Q \xrightarrow{l} Q' \quad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'}\ \mathrm{PAR}_2$$

$$\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l|l'} P' \mid Q'}\ \mathrm{SYN} \qquad \frac{P \xrightarrow{x[x']|y(y')} P'}{\nu xy P \xrightarrow{\tau} \nu xy\, \nu x'y'\, P'}\ \otimes\!\!\!\!\bigcirc$$

$$\frac{P \xrightarrow{x[]|y()} P'}{\nu xy P \xrightarrow{\tau} P'}\ 1\!\perp \qquad \frac{P \xrightarrow{l} P' \quad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\nu xy P \xrightarrow{l} \nu xy P'}\ \mathrm{RES} \qquad \frac{P =_\alpha Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'}\ =_\alpha$$

$$\mathrm{i}(l|l') = \mathrm{i}(l) \cup \mathrm{i}(l') \qquad \mathrm{f}(l|l') = \mathrm{f}(l) \cup \mathrm{f}(l')$$
$$\mathrm{i}(\tau) = \emptyset \qquad \mathrm{f}(\tau) = \emptyset$$
$$\mathrm{i}(x[]) = \mathrm{i}(x()) = \emptyset \qquad \mathrm{f}(x[]) = \mathrm{f}(x()) = \{x\}$$
$$\mathrm{i}(x[x']) = \mathrm{i}(x(x')) = \{x'\} \qquad \mathrm{f}(x[x']) = \mathrm{f}(x(x')) = \{x\}$$

Fig. 5. πMLL, transition rules for processes.

Remark 2.8. The untyped LTS for processes relies heavily on explicit side conditions regarding free and introduced names to prevent the collision of bound variables (e.g., ensuring $i(l) \cap i(l') = \emptyset$). However, when operating at the level of typing derivations, these side conditions become inherently redundant. Because πLL enforces linear resource management, the structural rules of the typed LTS dictate that hyperenvironments can only be merged when their domains are strictly disjoint. Consequently, any well-typed derivation inherently satisfies the name-distinctness and freshness requirements of the underlying untyped calculus, shifting the burden of safety from dynamic transition checks to static structural typing.

Proposition 2.9. *The operational semantics of πLL processes strictly respects the introduction of names via transition labels. Formally, if $P \xrightarrow{l} P'$, then $\mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset$, and If P is well-typed, then $\mathrm{i}(l) = \mathrm{f}(P') \setminus \mathrm{f}(P)$ (TODO: Evaluate relevance - delete depending on if this property is mentioend in later proofs).*

Remark 2.10. Proposition 2.9 is vital for mechanizing the metatheory of πLL. Because proof assistants tools require explicit tracking of fresh and bound variable to prevent accidental capture, this proposition acts as the formal bridge between the dynamic semantics and the static typing. It guarantees that a well-typed process evolves predictably, ensuring that it does not spontaneously generate resources during a transition. (TODO: same as Proposition 2.9)

2.7 Semantics of Typing Environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In πLL, types capture communication protocols. Consequently, the transition rules for typing environemnts specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how we defined the process erasure function, we again follow Montesi and Peressotti [2021] and define a function $env : Der \to Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, Montesi and Peressotti [2021] derive a LTS specifically for typing environments (lts_e), shown in 6.

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x : \perp$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is formalized by the τ-reflexivity axiom, $\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma$, which states that any valid hyperenvironment transitions to itself under a silent τ-action.

$$x : 1 \xrightarrow{x[]} \varnothing \qquad \Gamma, x : \bot \xrightarrow{x()} \Gamma \qquad \Gamma, \Delta, x : A \otimes B \xrightarrow{x[x']} \Gamma, x' : A \mid \Delta, x : B \qquad \Gamma, x : A \parr B \xrightarrow{x(x')} \Gamma, x' : A, x : B$$

$$\frac{\mathcal{G} \xrightarrow{l} \mathcal{G}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G}' \mid \mathcal{H}}\ \text{PAR}_1 \qquad \frac{\mathcal{H} \xrightarrow{l} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G} \mid \mathcal{H}'}\ \text{PAR}_2 \qquad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}' \quad \mathcal{H} \xrightarrow{l'} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l|l'} \mathcal{G}' \mid \mathcal{H}'}\ \text{SYN}$$

$$\frac{\mathcal{G} \mid \Gamma, x : A^\perp \mid \Delta, y : A \xrightarrow{l} \mathcal{G}' \mid \Gamma', x : A^\perp \mid \Delta', y : A}{\mathcal{G} \mid \Gamma, \Delta \xrightarrow{l} \mathcal{G}' \mid \Gamma', \Delta'}\ \text{RES} \qquad \frac{\mathcal{G} \mid x : 1 \mid \Gamma, y : \bot \xrightarrow{x[]|y()} \mathcal{G} \mid \Gamma}{\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma}\ 1\bot$$

$$\frac{\mathcal{G} \mid \Gamma, \Delta, x : A \otimes B \mid \Xi, y : A^\perp \parr B^\perp \xrightarrow{x[x']|y(y')} \mathcal{G} \mid \Gamma, x : B \mid \Delta, x' : A \mid \Xi, y : B^\perp, y' : A^\perp}{\mathcal{G} \mid \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \otimes\parr$$

Fig. 6. πMLL, transition rules for typing environments.

The requirement that a process can only perform actions permitted by its typing environment is known as **Session Fidelity**. Just as the *proc* function established operational correspondence for processes, the *env* function establishes a correspondence for environments.

While the rules in Figure 6 define how environments can transition in isolation, the execution of a well-typed program requires the process and its environment to step in unison. This co-dependent relationship has processes only perform actions permitted by its typing environment, while the environment dynamically mirrors the physical control flow of the process, and is formally captured by the property of **Session Fidelity**. Just as the *proc* function established operational correspondence for the untyped calculus, the *env* function establishes this exact behavioral correspondence for the typing contexts.

Theorem 2.11 (Session Fidelity). *Let $\mathcal{D}$ be a valid typing derivation such that $env(\mathcal{D}) = \mathcal{G}$ and $proc(\mathcal{D}) = P$. If the process transitions such that $P \xrightarrow{l} P'$, then the environment can mimic this transition $\mathcal{G} \xrightarrow{l} \mathcal{G}'$, and there exists a new valid derivation $\mathcal{D}'$ for P' under $\mathcal{G}'$.*

To illustrate Session Fidelity in action, we can observe how the environment is constrained to follow the exact execution traces of the process it types.

Example 2.12 (Session Fidelity in Execution). Recall the process derivation from Example 2.5, typed by the environment $\mathcal{G} = v : 1, w : \bot \mid y : 1$. The process begins by performing an observable input action on w. By Session Fidelity, the environment must faithfully mimic this action, consuming the corresponding type:

$$v : 1, w : \bot \mid y : 1 \xrightarrow{w()} v : 1 \mid y : 1$$

The resulting derivative of the process is now typed by this new environment. At this intermediate stage, if we were to view the environment LTS in isolation, the pure semantics would theoretically allow an output on y to fire immediately, as $y : 1$ is an available, unblocked resource.

However, Session Fidelity dictates a strict co-dependence. The environment merely mirrors the process, and the underlying process term strictly guards the $y[]$ action behind a prefix, requiring a prior τ-synchronization. Therefore, the environment must wait. When the process finally resolves its internal communication, the environment mimics it via the τ-reflexivity axiom, remaining structurally unchanged:

$$v : 1 \mid y : 1 \xrightarrow{\tau} v : 1 \mid y : 1$$

Only after this synchronization unblocks the continuation can the process perform the $y[]$ action, which the environment once again mirrors by consuming the final resource:

$$v : 1 \mid y : 1 \xrightarrow{y[]} v : 1$$

Remark 2.13 (The Mechanization Gap). Session Fidelity is the ultimate touchstone for validating the safety of a session-typed language. However, verifying this property in a proof assistant exposes a stark gap between paper mathematics and mechanized logic. On paper, the invariant that an environment *dynamically mirrors* a process is highly intuitive, and transitions are easily massaged using structural equivalences and implicit variable conventions. In a machine, however, formally proving that types are consumed correctly requires a rigorous, algorithmic representation of bound variables and explicit freshness. The theoretical elegance of the environment LTS cannot be mechanized without first establishing a concrete architectural foundation for these variables, which dictates the design choices discussed in the following chapters.

2.8 The Burden of Bound variables

Having established the operational semantics of πLL and the ultimate goal of verifying `Session Fidelity`, we must address a silent mathematical challenge that underpins this entire system: the management of bound variables. To prove that an environment correctly mimics a process's transitions, we must perform exact syntactic matching of channel names. However, variables are routinely bound during the restriction of channels ($\boldsymbol{\nu} xy\, P$) and the reception of names ($x(y).P$). The precise identity of these bound names is irrelevant to the behavior of the process; what matters is the structural relationship between the binder and its occurrences.

This principle is formalized as **α-equivalence**, which states that two terms are identical if they differ only by the renaming of their bound variables. For instance, the process $\boldsymbol{\nu} xw\,(x[].\mathbf{0} \mid w().\mathbf{0})$ is α-equivalent to $\boldsymbol{\nu} xy\,(x[].\mathbf{0} \mid y().\mathbf{0})$. In the operational semantics, the structural congruence rule (rule $=_\alpha$) explicitly relies on α-equivalence to rename bound variables on the fly. This ensures that as processes evolve and parallel components interact, restricted scopes do not accidentally capture or collide with existing free names in the environment.

When defining operations like substitution, preventing accidental capture requires explicitly defining freshness conditions. To substitute a name z for a free name x in a restricted process $\boldsymbol{\nu} yw\, P$, one must ensure that $z \neq y$ and $z \neq w$. If z is already in use, the bound variables y and w must be safely α-renamed to completely fresh names before the substitution can proceed.

In paper-based presentations of process calculi, managing these α-renamings and explicit freshness conditions is notoriously tedious. To avoid cluttering proofs with renaming arithmetic, authors universally adopt **Barendregt's Variable Convention**. This convention simply assumes that, at all times, the set of bound variables in a formula is chosen to be completely disjoint from the set of free variables, and that any newly introduced binder uses a universally fresh name.

While Barendregt's convention is an elegant and effective tool for human-readable mathematics, it is a purely informal meta-linguistic agreement. A proof assistant, which requires rigorous, algorithmic verification of every logical step, cannot simply *assume* that a variable is fresh. In a mechanized setting, explicitly tracking named variables, computing freshness conditions, and manually applying α-equivalence across parallel components introduces an overwhelming amount of proof boilerplate.

Consequently, the gap between the fluid, paper-based operational semantics of πLL and its concrete mechanization is defined largely by the burden of bound variables. To formalize the metatheory without being paralyzed by variable capture, we must abandon the informal Barendregt

convention and adopt a structural architecture that inherently solves the α-equivalence problem. The design and implementation of this architecture is the focus of the following chapter.

References

Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.

Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23-26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33

Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/archives/fall2023/entries/logic-linear/

Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4

Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.

Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π-calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818

Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.